PROJECT REPORT

SWIPEX

Swipe-Based Intelligent Job Discovery and Career Assistance Platform

Submitted in partial fulfilment of the requirements for the
AI-Focused Software Engineering Internship Evaluation

Technology Stack: React (Vite) | Tailwind CSS | Django | Django REST Framework | PostgreSQL | JWT

Abstract

SwipeX is a role-based web platform that brings together job discovery and recruitment management into a single system built around a swipe-based interaction model. The platform serves two distinct user roles, job seekers and recruiters, each with dedicated dashboards, workflows, and permissions. Job seekers browse and interact with job listings through a swipe interface, save or skip opportunities, apply for positions, upload resumes, and track the status of their applications. Recruiters create company profiles, post and manage job openings, and review and manage applicants through a structured pipeline.

The system is built on a modern web stack: a React front end (scaffolded with Vite and styled using Tailwind CSS) communicates with a Django back end through a REST API built with Django REST Framework. PostgreSQL serves as the persistent data store, and JSON Web Tokens (JWT) handle stateless authentication and role-based access control. On top of this core engineering foundation, SwipeX integrates a set of AI-assisted features that support, rather than replace, human decision-making: assisted job description drafting for recruiters, resume analysis, profile-to-job matching, job recommendations, a career assistant, and skill recommendations for job seekers.

This report documents the objectives, architecture, functional modules, AI-assisted features, database and security design, testing approach, and outcomes of the SwipeX project, along with the engineering challenges encountered during development and the scope for future enhancement.

Keywords

Recruitment Platform, Job Discovery, Swipe-Based Interaction, React, Django REST Framework, PostgreSQL, JWT Authentication, AI-Assisted Recommendations, Resume Analysis, Career Assistant, Role-Based Access Control.

1. Introduction and Problem Statement

1.1 Introduction

Online recruitment has become the dominant channel through which job seekers find opportunities and organisations find talent. Most existing platforms, however, are built around long, form-heavy listings and static search filters, a design that has changed little since the early days of job portals. SwipeX approaches the same problem from a different angle: it borrows the fast, low-friction browsing pattern popularised by swipe-based mobile applications and applies it to job discovery, while retaining the structured workflows that recruiters need to manage postings and applicants at scale.

The project was undertaken as a full-stack software engineering exercise, with artificial intelligence integrated as a supporting layer rather than as the central premise of the system. The emphasis throughout development was on building a working, role-based web application with clean separation between the front end, the API layer, and the data layer, and then adding assistance features that make the recruitment workflow faster and more informative for both sides of the platform.

1.2 Background

Job seekers today typically manage their search across multiple portals, each with a different interface, a different application process, and no shared context between them. Recruiters, on the other hand, are often dealing with a large and unfiltered volume of applications, and manual screening remains time-consuming and inconsistent. There is a genuine gap for a platform that lowers the effort required from job seekers to discover relevant openings, while giving recruiters a workflow that keeps postings organised and applicants easy to review.

1.3 Problem Statement

Existing recruitment platforms tend to fall into one of two categories: consumer-facing job boards that are easy to browse but offer little structure for recruiters, or enterprise applicant tracking systems that are powerful for recruiters but not designed with a fast, engaging discovery experience for job seekers. Few platforms combine a lightweight, swipe-based discovery experience for candidates with a structured, dashboard-driven management workflow for recruiters, and fewer still layer assistance features on top of that combination in a way that is transparent about what the system does and does not decide.

1.4 Motivation

The motivation behind SwipeX was to design a platform where the job seeker experience is quick and low-friction, similar to browsing a feed, without losing the depth needed to search, filter, apply, and track applications meaningfully. On the recruiter side, the motivation was to give hiring teams a single place to post jobs, manage listings, and move through applicants without needing separate tools for each step. Introducing AI-assisted features on top of this core workflow was intended to reduce repetitive manual work, such as drafting job descriptions from scratch or manually comparing every resume against every listing, while keeping the final decisions with the human user.

1.5 Why a Smarter Recruitment Platform Is Required

As the volume of job postings and applications continues to grow, both job seekers and recruiters benefit from a system that reduces manual effort at each step of the process. For job seekers, this means faster discovery of relevant openings and clearer visibility into the status of their applications. For recruiters, this means less time spent on repetitive drafting and initial screening, and more time spent on evaluating genuinely promising candidates. SwipeX was designed around this principle: use straightforward, well-engineered software to handle the structure of the recruitment process, and use AI assistance selectively, at the points where it adds the most practical value, to support the people using the system rather than to replace their judgement.

2. Objectives and Scope

2.1 Project Objectives

- Design and build a role-based recruitment platform serving two distinct user types: job seekers and recruiters.
- Implement a swipe-based interaction model for job discovery that allows job seekers to quickly save, skip, or apply to listings.

- Provide job seekers with search, filtering, resume upload, and application tracking capabilities.
- Provide recruiters with tools to create company profiles, post and manage job listings, and manage applicants through a review pipeline.
- Integrate a set of AI-assisted features, including resume analysis, profile-to-job matching, job recommendations, a career assistant, skill recommendations, and assisted job description drafting, as a decision-support layer.
- Build the system on a modern, maintainable technology stack with a clear separation between the front end, the API, and the database.
- Implement secure, stateless authentication and role-based access control across the application.

2.2 Scope

The scope of SwipeX is defined around the two user roles that the platform serves. Each role has a dedicated set of features, and the system enforces role-based access so that job seeker and recruiter functionality remain cleanly separated at both the interface and the API level.

2.3 Job Seeker Scope

- Account registration, login, and a personal dashboard summarising activity and recommendations.
- Profile management, including skills and resume upload.
- Job discovery through search, filters, and a swipe-based interface, with saving and skipping of listings.
- Applying to jobs and tracking application status.
- Resume analysis, job/profile recommendations, career assistance, and skill recommendations.

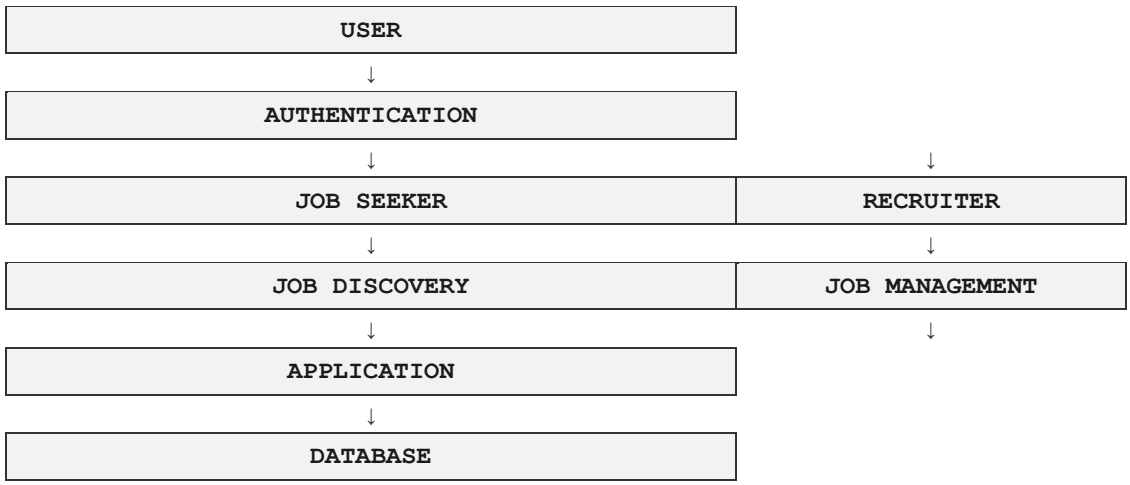
2.4 Recruiter Scope

- Account registration and login, with a dedicated recruiter dashboard and company profile management.
- Posting and managing job listings.
- Viewing and managing applicants for each posting.
- AI-assisted drafting support for job descriptions.
- Access to recruitment-related notifications and summary analytics where implemented.

2.5 System Overview

At a high level, SwipeX is a single web application with two role-specific experiences layered on top of a shared authentication and data model. A user registers and logs in, is authenticated through JWT, and is then routed to either the job seeker or the recruiter side of the platform based on their role. Both sides interact with the same underlying PostgreSQL database through the Django REST Framework API, ensuring that job listings, applications, and profile data remain consistent across the system.

Figure 2.1 — High-Level System Flow



3. System Architecture and Technology Stack

3.1 Overall Architecture

SwipeX follows a standard three-tier web architecture, with a clear boundary between the presentation layer, the application/API layer, and the data layer. This separation keeps the front end and back end independently deployable and testable, and allows each layer to evolve without requiring changes throughout the rest of the system.

3.2 Frontend

The client application is built using React, scaffolded with Vite for fast development builds and hot module reloading. Styling is implemented using Tailwind CSS, which allows the interface, including the swipe-based job discovery view, the job seeker and recruiter dashboards, and the various forms, to be built consistently using utility classes rather than hand-written stylesheets.

3.3 Backend

The server side is built using Django, which handles the application's business logic, request routing, and data models. Django's ORM manages the mapping between Python model classes and the underlying PostgreSQL tables, covering entities such as users, profiles, companies, job listings, and applications.

3.4 REST API

Django REST Framework (DRF) is used to expose the application's functionality as a REST API consumed by the React front end. DRF serializers handle validation and conversion between JSON payloads and Django model instances, and its view classes implement the endpoints used for authentication, job listing management, applications, and profile operations.

3.5 Database

PostgreSQL is used as the relational database for the platform. It stores user accounts, job seeker and recruiter profiles, company information, job postings, and application records, along with the relationships between them.

3.6 Authentication

Authentication is implemented using JSON Web Tokens (JWT). On successful login, the API issues a token that the client includes with subsequent requests, allowing the back end to identify the user and their role without maintaining server-side session state. Role information carried with the authenticated user is used to enforce access control between job seeker and recruiter functionality.

3.7 Deployment

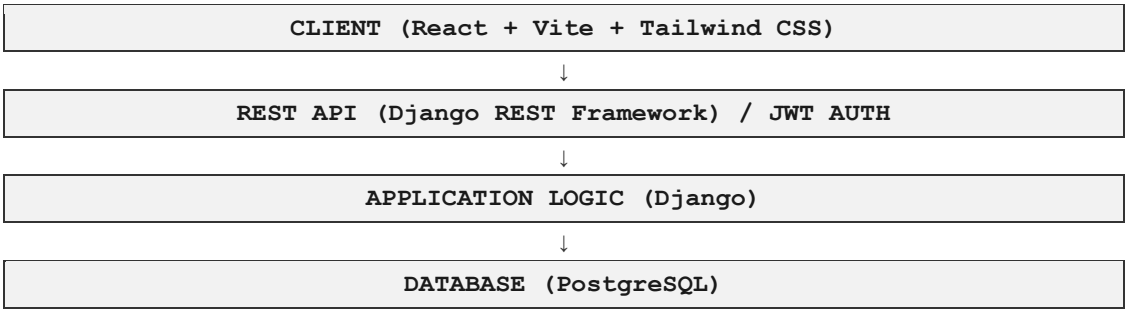
The front end is deployed separately from the back end, with the React application built and served through a static hosting/deployment platform such as Vercel, while the Django API and PostgreSQL database are deployed on a separate backend hosting environment. Version control for the project is managed using Git, with the repository hosted on GitHub.

3.8 Technology Stack Summary

Layer	Technology	Purpose
Frontend	React + Vite	Component-based user interface and fast development builds
Styling	Tailwind CSS	Utility-first styling for consistent, responsive design
Backend	Django	Application logic, data models, request handling
API	Django REST Framework	REST endpoints consumed by the frontend
Database	PostgreSQL	Persistent relational data storage
Authentication	JWT	Stateless authentication and role-based access
Version Control	Git / GitHub	Source code management and collaboration
Deployment	Vercel (frontend) + backend hosting	Hosting and delivery of the application

Table 3.1 — Technology Stack

Figure 3.1 — Architecture Overview



4. User Roles and Job Seeker Module

4.1 Role-Based Access

SwipeX defines two user roles at the point of registration: job seeker and recruiter. The assigned role determines which dashboard the user is routed to after login and which set of API endpoints they are permitted to access. Role checks are enforced on the backend, so that a job seeker account cannot access recruiter-only operations such as posting jobs or managing applicants, and vice versa.

4.2 Job Seeker Dashboard

After logging in, a job seeker is presented with a dashboard summarising their activity: recently viewed or saved jobs, the status of their current applications, and any recommendations generated for their profile. The dashboard acts as the central starting point from which the seeker can move into job discovery, profile management, or application tracking.

4.3 Profile Management

Job seekers maintain a profile containing their personal details, listed skills, and an uploaded resume. This profile forms the basis for the platform's matching and recommendation features described in Section 6, and can be updated at any time as the seeker's skills or resume change.

4.4 Job Discovery

The core discovery experience is built around a swipe-based interface: job listings are presented one at a time in a card-style layout, and the seeker can act on each card, saving it for later, skipping it, or moving directly into the application flow. This is complemented by conventional search and filter controls, allowing seekers who prefer a more targeted search to narrow listings by criteria such as role, location, or keyword.

4.5 Search and Filtering

Alongside the swipe interface, job seekers can search listings directly and apply filters to narrow results. This gives the seeker two complementary ways to browse the same underlying job data: a fast, low-effort swipe mode for general exploration, and a more deliberate search mode when looking for something specific.

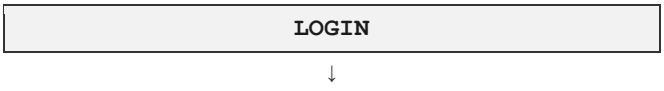
4.6 Saved Jobs and Applications

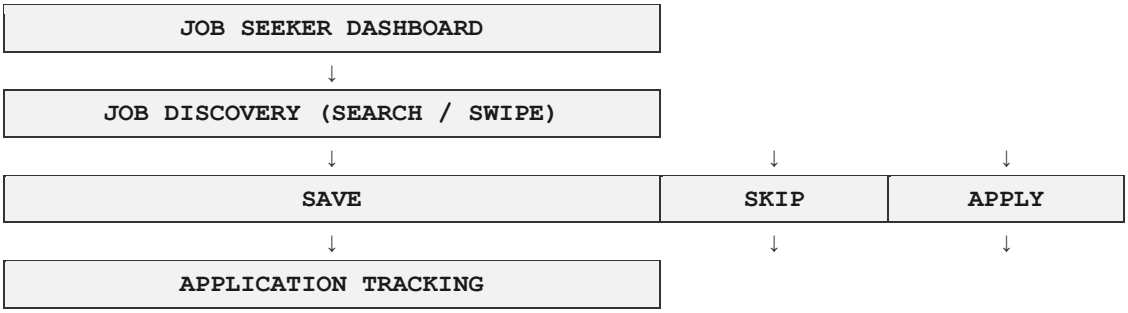
Jobs that a seeker saves while swiping are collected in a dedicated saved-jobs list for later review. When the seeker chooses to apply, the system records an application entry linked to their profile and the job listing, and the seeker can track the status of each application from their dashboard as it progresses through the recruiter's review pipeline.

4.7 Resume Upload

Job seekers upload a resume as part of their profile. The uploaded resume is stored and associated with the seeker's account, and is used both when applying to jobs and as an input to the resume analysis feature described in Section 6.

Figure 4.1 — Job Seeker Workflow





5. Recruiter Module

5.1 Recruiter Dashboard

The recruiter dashboard gives a hiring user an overview of their active job postings, recent applicants, and any pending actions such as applications awaiting review. It serves as the entry point into the recruiter-specific parts of the platform, in the same way the job seeker dashboard does for candidates.

5.2 Company Profile

Recruiters maintain a company profile that is displayed alongside their job postings, giving job seekers basic context about the organisation behind a listing. This profile is created and edited from the recruiter dashboard and is shared across all postings made under that recruiter account.

5.3 Job Posting

Recruiters create new job listings through a structured posting form covering fields such as role title, description, required skills, and location. Once published, a listing becomes visible to job seekers through both the search interface and the swipe-based discovery feed.

5.4 Job Management

Existing postings can be edited, updated, or closed from the recruiter dashboard. This keeps listings accurate over time, for example when a role's requirements change or a position is filled and needs to be removed from active discovery.

5.5 Applicant Management

For each job posting, recruiters can view the list of job seekers who have applied and move each applicant through a review pipeline. This gives recruiters a structured way to track where each candidate stands, rather than managing applications through email or spreadsheets.

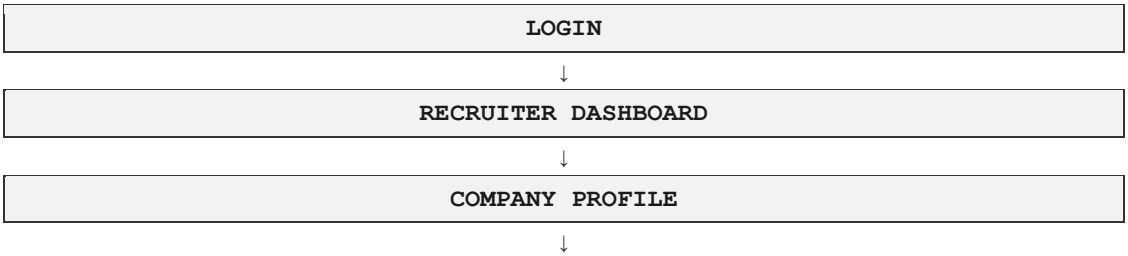
5.6 Application Review

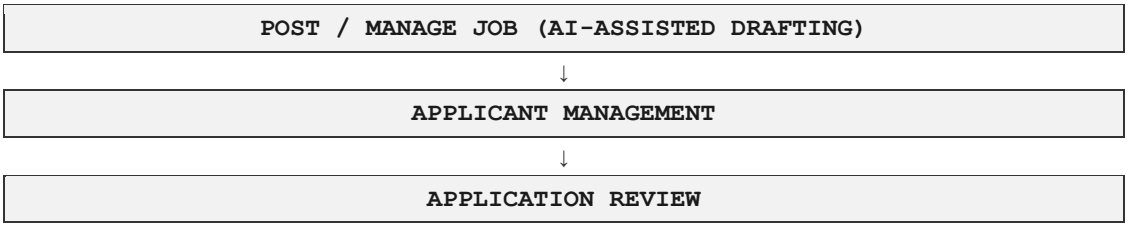
Reviewing an application gives the recruiter access to the applicant's profile, skills, and uploaded resume, along with any resume analysis or match information generated for that job seeker in relation to the specific listing. The recruiter uses this information to decide how to proceed with each applicant; the platform presents supporting information but does not make the accept or reject decision on the recruiter's behalf.

5.7 AI-Assisted Job Description Drafting

When creating a new listing, recruiters can use an AI-assisted drafting feature to generate a starting draft of the job description based on basic inputs such as the role title and required skills. The recruiter reviews and edits this draft before publishing, so the feature functions as a writing aid that reduces the effort of starting from a blank page, rather than an automated posting system.

Figure 5.1 — Recruiter Workflow





6. Artificial Intelligence Features

AI features in SwipeX are designed as a decision-support layer positioned around the core recruitment workflow. Each feature takes existing profile, resume, or job data as input, produces an output intended to inform the user, and leaves the final decision, whether to apply, whom to shortlist, or how to phrase a posting, with the person using the platform. None of the AI features described below make an autonomous hiring decision or independently reject or accept a candidate; they surface information and suggestions that a job seeker or recruiter can act on.

6.1 AI-Assisted Job Description Generation

Input: role title, required skills, and any additional notes supplied by the recruiter. Processing/Assistance: the system generates a draft description structured around the supplied inputs. Output: an editable draft job description. Benefit: reduces the time recruiters spend writing postings from scratch and provides a consistent starting structure across listings.

6.2 Resume Analysis

Input: the resume uploaded by the job seeker. Processing/Assistance: the system extracts relevant information from the resume, such as listed skills and experience, to build a structured view of the candidate's profile. Output: a summarised, structured representation of the resume's content. Benefit: gives both the job seeker and, where relevant, the recruiter a quick, organised view of the resume's content without requiring a full manual read-through.

6.3 Job/Profile Matching

Input: the job seeker's profile and skills, together with the requirements of a given job listing. Processing/Assistance: the system compares the two to estimate how closely a candidate's profile aligns with a specific listing. Output: a relative indication of fit between the profile and the job. Benefit: helps job seekers prioritise which listings to pursue and helps recruiters get a quick sense of applicant relevance.

6.4 Job Recommendations

Input: the job seeker's profile, skills, and past interactions such as saved or applied jobs. Processing/Assistance: the system uses this information to surface listings likely to be relevant to the seeker. Output: a personalised list of suggested job openings. Benefit: reduces the effort of manually searching through all available listings by surfacing the most relevant ones first.

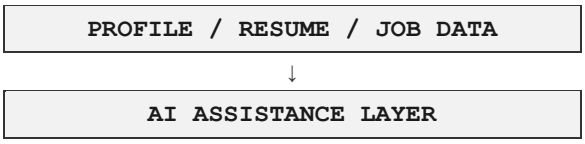
6.5 Career Assistant

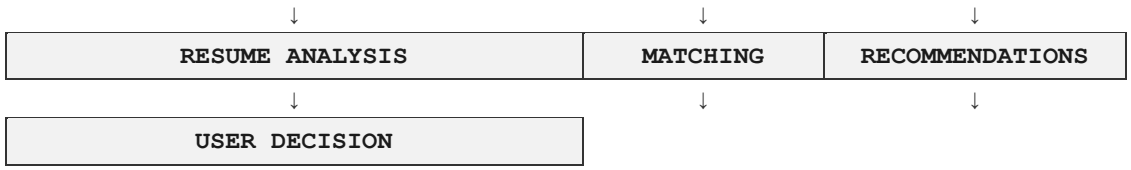
Input: the job seeker's profile and stated goals or queries. Processing/Assistance: the assistant provides guidance related to career progression, such as suggestions on profile completeness or general direction based on the seeker's stated interests. Output: conversational or summarised career guidance. Benefit: gives job seekers a point of reference for career-related questions within the platform itself.

6.6 Skill Recommendations

Input: the job seeker's current listed skills and the requirements observed across relevant job listings. Processing/Assistance: the system identifies gaps between the seeker's current skill set and commonly required skills for roles they are interested in. Output: a list of suggested skills to develop. Benefit: gives job seekers concrete, actionable direction for improving their fit for the roles they are targeting.

Figure 6.1 — AI Assistance Workflow





Across all six features, the underlying principle is the same: AI assistance is used to organise, summarise, or surface information that would otherwise require manual effort to compile, while the interpretation of that information and the resulting decision remain with the job seeker or recruiter.

7. Database, API and Security

7.1 PostgreSQL

SwipeX uses PostgreSQL as its relational database, managed through Django's ORM. PostgreSQL was chosen for its reliability, strong support for relational integrity constraints, and straightforward integration with Django, which handles schema migrations as the data model evolves.

7.2 Main Entities

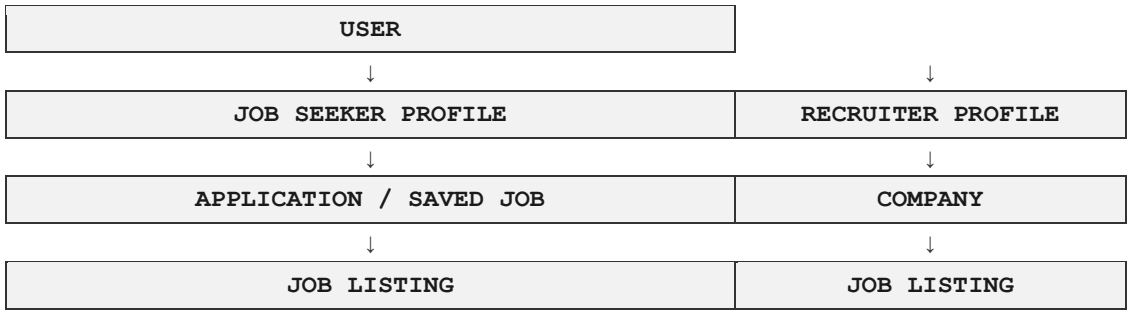
Entity	Description
User	Base account record with role (job seeker or recruiter), credentials, and profile link
Job Seeker Profile	Skills, resume reference, and preferences associated with a job seeker account
Recruiter Profile	Recruiter account details linked to a company
Company	Company information displayed alongside job postings
Job Listing	Job postings created by recruiters, including title, description, and required skills
Application	Links a job seeker to a job listing, with status tracked through the review pipeline
Saved Job	Record of jobs a job seeker has saved from the discovery feed

Table 7.1 — Main Database Entities

7.3 Relationships

A User record extends into either a Job Seeker Profile or a Recruiter Profile, depending on role. A Recruiter Profile is linked to a Company, and a Company can have multiple Job Listings. Each Job Listing can receive multiple Applications, and each Application links exactly one Job Seeker Profile to one Job Listing. Saved Jobs similarly link a Job Seeker Profile to one or more Job Listings without creating a formal application.

Figure 7.1 — Entity Relationship Overview



7.4 Django REST Framework and API Communication

The React front end communicates with the Django back end exclusively through REST endpoints exposed via Django REST Framework. Requests and responses use JSON, with DRF serializers handling validation on the way in and consistent formatting on the way out. Endpoints are grouped by resource, covering authentication, profiles, job listings, applications, and the AI-assisted features described in Section 6.

7.5 JWT Authentication and Role-Based Access

Authentication is handled using JSON Web Tokens issued at login. Each subsequent request from the client includes the token, which the backend verifies to identify the user and their role before processing the request. Role-based access control is enforced at the API level, so that job seeker and recruiter endpoints remain separated regardless of what the front end displays.

7.6 Environment Variables

Sensitive configuration values, including database credentials, JWT signing keys, and any third-party service keys, are kept out of the codebase and managed through environment variables, loaded separately in the development and deployment environments.

7.7 Security Considerations

- Passwords are stored using Django's built-in hashing mechanisms rather than in plain text.
- JWT tokens carry a limited lifetime to reduce the impact of token exposure.
- Role checks are enforced server-side, not left to the front end alone.
- Input validation is handled through DRF serializers before data reaches the database.
- Environment-specific secrets are excluded from version control.

8. Testing, Technical Challenges and Results

8.1 Testing Approach

Testing of SwipeX was carried out across the main functional areas of the platform, covering both individual features and the integration between the front end and back end.

- Authentication testing: verifying registration, login, token issuance, and rejection of invalid or expired credentials.
- Recruiter and job seeker workflow testing: covering company profiles, job posting and editing, applicant management, profile setup, resume upload, discovery, and applying.
- Application testing: confirming applications are correctly recorded and that status updates propagate to the job seeker's dashboard.
- AI feature testing: checking that resume analysis, matching, recommendations, the career assistant, skill recommendations, and job description drafting return sensible output.
- Frontend/backend integration testing: confirming API responses are correctly consumed and rendered across both user roles.

8.2 Technical Challenges

Development of SwipeX surfaced a number of practical engineering challenges, consistent with building a full-stack, role-based platform from the ground up.

- Database and model synchronisation: keeping Django models, migrations, and the PostgreSQL schema aligned as the data model evolved, particularly around profile, listing, and application relationships.
- Frontend/backend integration: coordinating API contracts between React and DRF, especially for the swipe-based discovery interface and its sequential job data.
- Role-based access: consistently enforcing access control across every endpoint so role separation could not be bypassed from the client side.
- AI feature integration: fitting AI-assisted features into the existing request/response flow while keeping response times and output formats consistent.
- Deployment and environment configuration: managing separate frontend/backend deployment targets and correctly configuring environment variables and CORS.

These challenges were addressed as normal parts of the development process rather than as blockers: model and schema issues were resolved through careful migration management, integration issues through clearer API contracts and testing, and deployment issues through incremental configuration and verification across environments.

8.3 Results

Module	Status	Notes
Authentication (JWT)	Working	Registration, login, and role-based routing verified
Job Seeker Dashboard	Working	Profile, saved jobs, and applications displayed correctly
Job Discovery (Swipe)	Working	Swipe interactions correctly recorded as save/skip/apply
Search and Filters	Working	Listings correctly filtered by supplied criteria
Recruiter Dashboard	Working	Company profile and job postings managed successfully
Applicant Management	Working	Applicants correctly listed and tracked per job posting
AI-Assisted Features	Working	Drafting, analysis, matching, and recommendations return usable output
Deployment	Working	Frontend and backend deployed and communicating correctly

Table 8.1 — Summary of Testing Results

9. Conclusion and Future Scope

9.1 Conclusion

SwipeX demonstrates a complete, role-based recruitment platform built on a modern full-stack architecture, combining a React front end, a Django REST Framework API, a PostgreSQL database, and JWT-based authentication. The project delivers a working swipe-based job discovery experience for job seekers alongside a structured job posting and applicant management workflow for recruiters, with the two roles cleanly separated at both the interface and the API level.

9.2 Achievements

- A fully functional, role-based web application supporting distinct job seeker and recruiter experiences.
- A swipe-based job discovery interface integrated alongside conventional search and filtering.
- A structured recruiter workflow covering company profiles, job postings, and applicant management.
- Secure, stateless authentication with role-based access control enforced at the API level.
- Integration of AI-assisted features as a supporting layer across both job seeker and recruiter workflows.

9.3 AI Contribution

Across the platform, AI assistance reduced manual effort at specific points: helping recruiters start job descriptions faster, giving job seekers a structured view of their resume, estimating fit between profiles and listings, surfacing relevant recommendations, offering career guidance, and identifying skill gaps. Each feature was designed to inform the user's decision rather than make it automatically, consistent with treating AI as a decision-support layer rather than an autonomous hiring mechanism.

9.4 Limitations

- Matching and recommendation quality depends on the structured data available in a profile or listing.
- The career assistant offers general guidance rather than personalised counselling from external labour-market data, and AI-assisted drafting still requires recruiter review before publishing.

9.5 Future Scope

- Semantic job matching and natural language search, going beyond keyword overlap so seekers can describe what they want in plain language.
- Improved resume-to-job matching as usage data grows, plus AI-assisted interview preparation support.
- Personalised learning paths, more advanced recruiter analytics, and explainable recommendations.

- A real-time notifications system and a dedicated mobile application extending the swipe-based experience.

Taken together, these directions extend SwipeX toward a more capable and transparent recruitment assistance platform, while preserving the principle established in this project: AI supports the people using the platform, and the people remain in control of the decisions that matter.